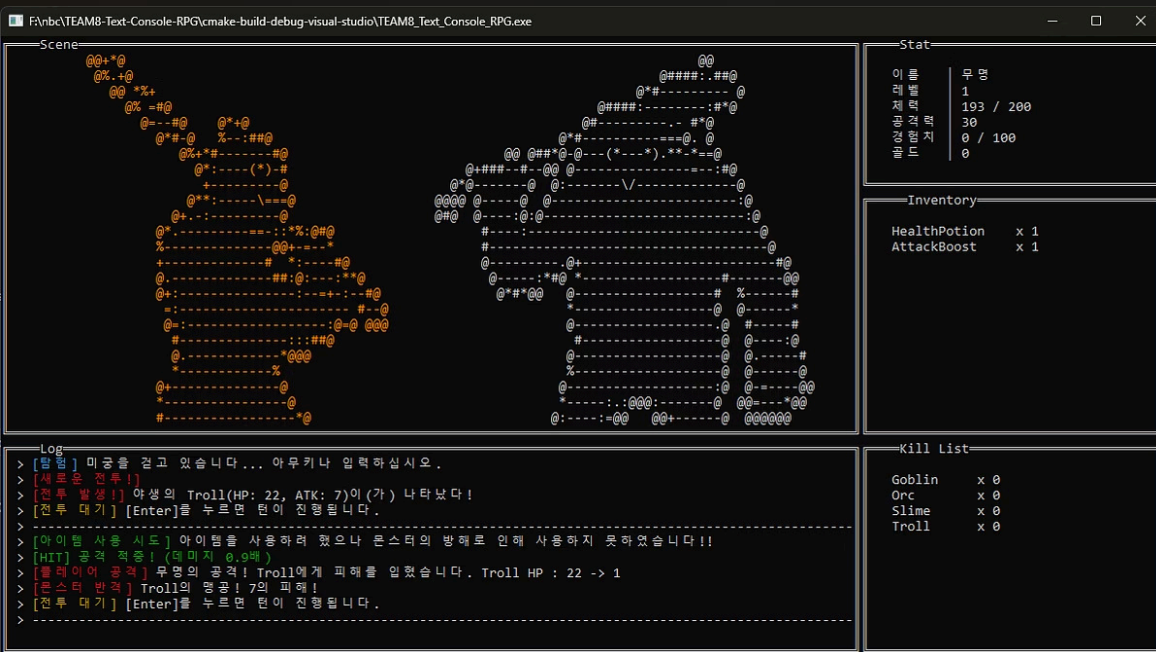




8조

Auto Battler Text RPG

게임 개발 발표

프로젝트 개요

주요 특징

장르

- Auto Battler + Text RPG의 구조
- ASCII ART
- Perfect Zone Mini Game

화면 설계

단일 화면을 분할하여 로그, 캐릭터 상태, 인벤토리, 전투 장면의 정보를 한눈에 확인 가능

시스템 구조

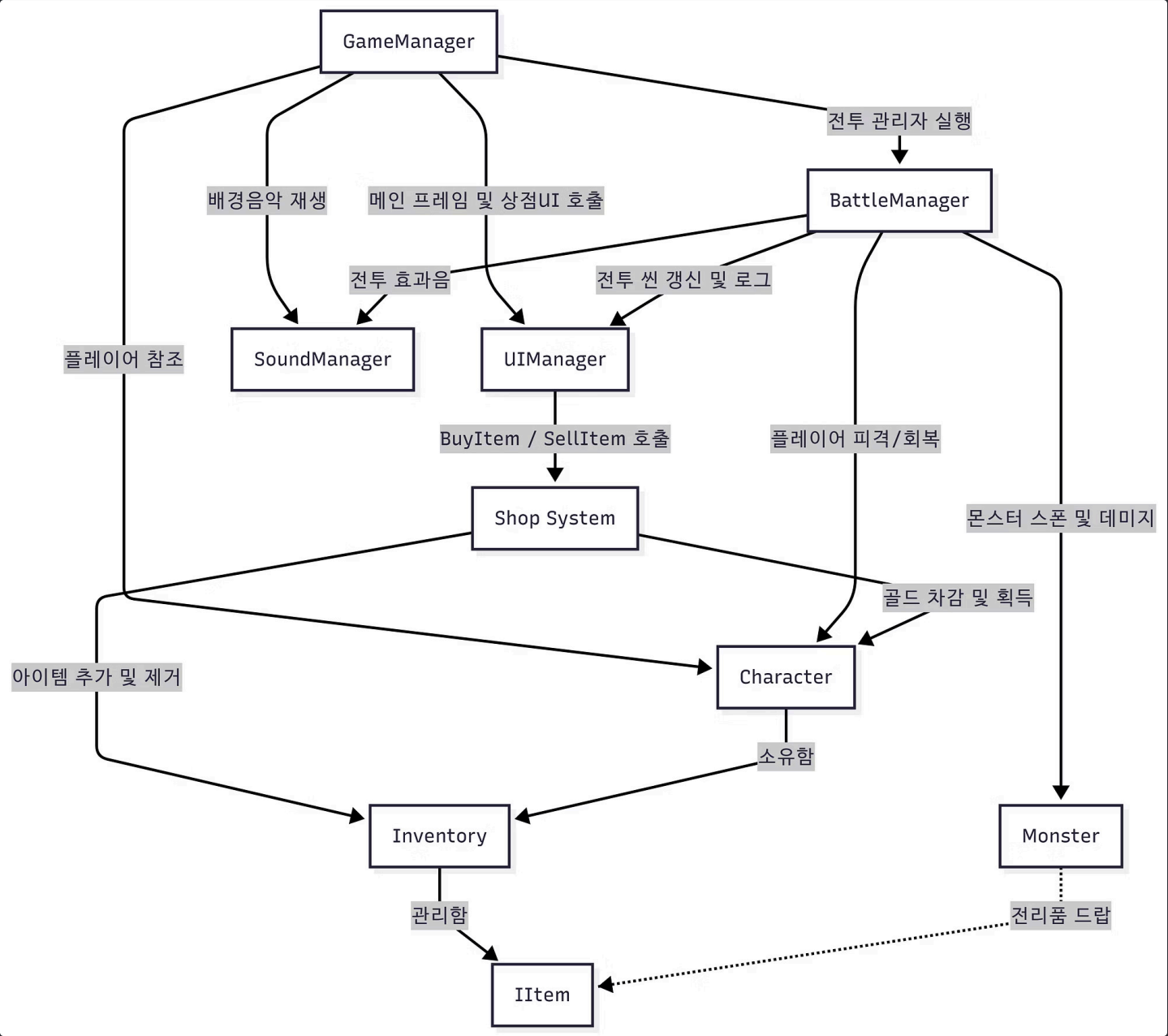
GameManager를 중심으로 Item, Inventory, Character, Monster, UI, Shop, Sound 시스템이 연동

게임 메인 루프 (영상)

- 1 — 전투 진입
- 2 — 아이템 랜덤 사용 (30%)
- 3 — 플레이어 공격 and 몬스터 죽음 체크
- 4 — 몬스터 공격 and 플레이어 죽음 체크
- 5 — 상점 입장

시스템 아키텍처

프로젝트 구조



조민웅

- GameManager
- BattleManager

이승민

- UIManager
- SoundManager

김진우

- Character

김선우

- Monster
- Shop System

이인구

- Inventory
- Item

팀 협업 & 학습 과정

프로젝트를 진행하며 팀원들은 단순한 게임 개발을 넘어, 효과적인 협업 방식과 시스템 설계 원칙을 학습하고 적용했습니다.

1 Github 협업 학습

팀원 모두 Github를 처음 사용하는 상황. 단순히 게임을 만드는 것뿐 아니라, 협업을 할때에 발생하는 문제들에 대해서 다양하게 겪어보는 것을 핵심 목표로 설정했습니다.

2 초기 아키텍처 설계

프로젝트 시작 전 Notion에서 UML 다이어그램을 사용. 이를 바탕으로 GameManager 중심의 시스템 구조를 설계했습니다.

3 매니저 모듈 분리

초기 GameManager에 모든 로직이 집중되어 있었음. 개발 진행 중 BattleManager를 분리하여 전투 로직 독립화. UIManager를 추가하여 화면에 그리는 모든 것들을 중앙에서 관리했습니다.

4 SoundManager 통합

게임의 완성도를 높이기 위해 최종 단계에서 사운드 시스템을 통합했습니다.

5 QA

레벨 디자인 및 수치 조정

트러블 슈팅

화면 깜빡임 개선: 부분 갱신(Partial Redraw) & 메모리 버퍼링

문제 상황

기존의 `system("cls")` 명령어를 자주 사용할 경우, 콘솔 화면 전체가 매번 지워지고 다시 그려져 **극심한 화면 깜빡임 (Flickering) 현상**이 발생하여 사용자 경험을 저해했습니다.

해결 방안

화면 전체를 지우는 대신 `Gotoxy`를 활용한 좌표 기반의 부분 화면 갱신(Partial Redraw) 기법을 적용하여, 값이 변경된 특정 구역의 텍스트만 덮어쓰도록 구현

추가적으로 메모리 단에서 `std::string`을 이용해 완성된 문자열을 미리 조립한 후 한 번에 화면에 출력하는 메모리 버퍼링 (Memory Buffering) 기법을 결합하여 렌더링 최적화 아키텍처를 구현

기술적 성과

이러한 부분 갱신 최적화는 모던 3D 엔진(예: UE5의 UMG)에서도 핵심 개념으로 사용되는 **"변경된 요소만 다시 그린다 (Invalidation Box)"** 원리와 동일

트러블 슈팅

빌드 시스템 전환: .vcxproj → CMakeLists.txt

문제 상황

파일을 추가할 때마다 .vcxproj 파일에서 병합 충돌이 반복적으로 발생하여 협업 효율이 크게 저하되었습니다.

해결 방안

빌드 시스템을 **CMakeLists.txt**로 전환하여 플랫폼 독립적인 빌드 구성을 확보하고, 파일 추가 시 충돌 문제를 근본적으로 해소했습니다.

트러블 슈팅

Branch Rules & Git Convention

Branch Rules — 2인 승인 제도

Repo 초기 설정에는 한명의 개발자가 승인하는 구조였는데, 이 구조의 위험성을 인지하고, **최소 2명 이상의 승인**이 필요하도록 Branch Protection Rule을 설정했습니다.

① 코드 리뷰 문화 정착 및 실수 방지

Git Convention — 대소문자 구분 X

Branch 이름을 대소문자를 구분하여 짓다가 이름 충돌 문제가 발생했습니다. 이후 **소문자 통일 규칙**을 도입하여 충돌을 방지하고 일관성을 확보했습니다.

① 예: `feature/add-inventory` (소문자 + 하이픈)



마무리 멘트

감사합니다!

“

이승민

QA랑 폴리싱 과정이 디따 재밌네요

”

“

김진우

재밌있는 시간이었습니다. 덕분에 많이 배워 갑니다.

”

“

조민웅

재밌있는 시간이었습니다 클로드야 고마워!

”

“

김선우

웅ㅇ애 CPP 어려웠

”

“

이인구

고생 많으셨습니다, 교수님들 덕분에 꿀 빨다 갑니다. 감사합니다.

”